SectorQ: Highly Scalable Pauli Propagation on HPC Systems with Window-Conserved Parity Sectors

ANONYMOUS AUTHOR(S), Submission under review, Anonymous

This paper presents *SectorQ*, a Pauli propagation system designed to compute the expectation values of quantum circuits with more than 100 qubits on GPU clusters. As the accuracy of Pauli propagation grows with the number of kept terms, the terms of a 127-qubit circuit exceed the memory of hundreds of GPUs, and the existing multi-GPU engine merges them across all GPUs after every gate. *SectorQ* introduces window-conserved parity sectors, which divide the terms by the parities that every gate of a window leaves unchanged and keep each new term in the same sector as its merge partner. With this design, *SectorQ* applies each window without exchanging terms between nodes, balances the sectors by choosing the parities at run time from a sample of the live terms, and holds the sectors in host memory beyond the capacity of the GPU memory. Our evaluation using seven circuits on up to 512 NVIDIA A100 GPUs shows that *SectorQ* reduces the simulation time of *cuPauliProp* on 16 GPUs by 4.4-9.7× and increases the largest finished problem on 512 GPUs by 140×.

1 Introduction

Quantum computers have advanced from demonstrations of quantum supremacy on sampling tasks to processors with more than 100 qubits that measure the expectation values of physical models [3, 5]. These expectation values are the output of quantum simulation, the application that motivates quantum computing in physics, chemistry, and materials science [TODO: cite]. A 127-qubit processor, for example, measured the magnetization of a kicked Ising model at a scale beyond the exact classical methods [5]. This scale moves the comparison between quantum and classical computation from small benchmarks to the circuits of these applications. As a result, the range of expectation values that a classical computer can still produce has become a central question for the development of quantum computers.

Despite this progress, current processors remain noisy intermediate-scale quantum (NISQ) devices [8]. The noise of these devices biases every measured expectation value, and error mitigation recovers an estimate from repeated runs at amplified noise [5]. These estimates require an independent reference, and classical simulation on high-performance computing (HPC) systems provides this reference without noise. This reference verifies the hardware, calibrates the error mitigation, and tests quantum algorithms before they run on a processor. Thus, several groups simulated the 127-qubit experiment soon after its publication, with tensor networks and with Pauli propagation [4, 10].

Classical simulation of these circuits follows three approaches on HPC systems. State vector simulation stores all 2^n amplitudes of an n-qubit state, and this memory limits it to about 50 qubits on the largest systems [TODO: cite]. Tensor network simulation stores the state as a network of small tensors, and its cost grows with the entanglement of the circuit [TODO: cite]. Pauli propagation tracks the observable, not the state, as a list of Pauli terms, and its cost follows the number of kept terms in place of the number of qubits [4, 9]. This cost extends Pauli propagation to circuits with more than 100 qubits, and NVIDIA and IBM now provide Pauli propagation in their quantum software stacks [7] [TODO: cite Qiskit]. Our work focuses on Pauli propagation, since its accuracy follows the resources of the system in place of the number of qubits.

The accuracy of Pauli propagation, however, requires far more terms than one GPU holds. A lower cutoff on the coefficients keeps more terms, and the terms grow within each layer of gates before the truncation removes the small ones. For example, the 127-qubit kicked Ising circuit holds up to 1.4 billion terms within a layer at cutoff 10^{-5} and 100 billion terms at cutoff 10^{-6}, the

Author's Contact Information: Anonymous Author(s), Submission under review, Anonymous.

Table 1. Comparison with existing Pauli propagation systems (MN: Multi-Node, GPU: GPU Execution, LP: Locality-Preserving Partition, RP: Run-Time Partition from the Gates and the Live Terms, HM: Host Memory beyond GPU Memory)

System	Method	MN	GPU	LP	RP	HM
PauliPropagation.jl [9]	Pauli propagation, one node					
SPD [4]	Sparse Pauli dynamics		✓			
ORQA [2]	Distributed strings by hash	✓				
MonoProp [1]	Distributed Pauli propagation	✓				
cuPauliProp [7]	Global merge after every gate	✓	✓			
paulistrings-rs [6]	Fixed linear partition	✓	✓	✓		
***SectorQ* (Ours)**	**Window-conserved parity sectors**	✓	✓	✓	✓	✓

memory of about 14 and 1,000 GPUs. These counts place Pauli propagation on HPC systems, and recent systems run it on GPU clusters and on the Fugaku supercomputer with 2^{17} processes [2, 7]. However, the existing systems show limited performance and scalability as the number of terms grows.

The existing systems have two limits, and the first is the global step after every gate. Since a new term may merge with a term on any GPU or process, these systems exchange the new terms of every gate across all GPUs or processes [1, 2, 7]. This step adds a fixed cost to every gate, from 5.9 ms on 4 GPUs to 29.8 ms on 16 GPUs in *cuPauliProp*. The second limit is the GPU memory. Between two truncations, every rotation branches the terms, and the list grows to several times its size at the truncation. The merge of these terms also requires working space on the same GPU. As a result, *cuPauliProp* holds about 100 million kept terms on a GPU with 36 GB, and paulistrings-rs keeps its device partitions on the GPUs by the same rule [6]. A deferred merge reduces the global steps, but the unmerged copies of equal strings fill this memory faster. To summarize, the existing systems bind the cost of every gate to the number of GPUs and the size of the problem to the GPU memory, and our work aims to remove both bounds.

Many previous studies, as shown in Table 1, have optimized Pauli propagation from different algorithm and architecture perspectives. Single-node simulators reduce the cost of each gate with sparse data structures, symmetries of the circuit, and GPU kernels [4, 9] [TODO: cite]. Distributed CPU systems assign each Pauli string to one process by a hash or a distribution function and exchange the new strings after every gate [1, 2]. Distributed GPU engines keep the strings of each gate across all GPUs and merge them in a global step [7]. Linear partitions assign each string by parities of its bits, and these parities remain fixed for the whole run [6]. Theory on the commutator graph of a circuit also shows that the generators of the gates divide the Pauli strings into components that never mix [TODO: cite West et al.].

SectorQ distinguishes itself from previous studies by recomputing the partition of the terms for every window of gates during the run. Previous systems merge the terms globally after every gate or fix the partition before the run, and these choices bind the exchange to the gates and the balance to rows chosen in advance. In contrast, *SectorQ* derives the parities that each window conserves from the generators of its gates, chooses among them by the balance of a sample of the live terms, and assigns each sector to one node for the whole window. Each GPU loads one sector at a time from the host memory of its node, and the terms move between nodes only at window boundaries. This design removes the global step after every gate and the bound of the GPU memory, and enables scalable performance across circuits, problem sizes, and GPU counts.

In this paper, we present *SectorQ*, a Pauli propagation system for GPU clusters that partitions the terms by the parities that a window of gates conserves. A rotation gate maps each term to itself and to its product with the generator of the gate, and every parity that the generator leaves unchanged assigns the same value to both terms. Specifically, *SectorQ* introduces 1) window-conserved parity sectors that the system recomputes for every window from the generators of its gates, 2) a run-time choice of the parities and the number of sectors from a sample of the live terms while the GPUs process the current window, and 3) a host-memory sector store from which each GPU loads one sector at a time. With these sectors, the terms move between nodes only at window boundaries, and the number of terms that *SectorQ* holds follows the host memory of all nodes. Our evaluation on up to 512 NVIDIA A100 GPUs of the Perlmutter supercomputer shows that *SectorQ* reduces the simulation time by 4.4-9.7× on 16 GPUs and increases the largest finished problem by 140× on 512 GPUs compared with *cuPauliProp*, the existing multi-GPU engine. We open-source *SectorQ* at: https://anonymous.4open.science/r/SectorQ

2 Background

2.1 Pauli Propagation beyond 100 Qubits

Classical simulation of a quantum circuit stores either the state or the observable. State vector simulation stores all 2^n amplitudes of the state, and this memory limits it to about 50 qubits on the largest HPC systems [TODO: cite]. Tensor network simulation compresses the state into small tensors, and its cost grows with the entanglement of the circuit [10]. Pauli propagation tracks the observable, not the state, as a list of Pauli terms in the Heisenberg picture and computes its expectation value for circuits with more than 100 qubits [4, 9]. This list sets the cost of the simulation, and our work focuses on Pauli propagation, since its cost follows the number of kept terms in place of the number of qubits.

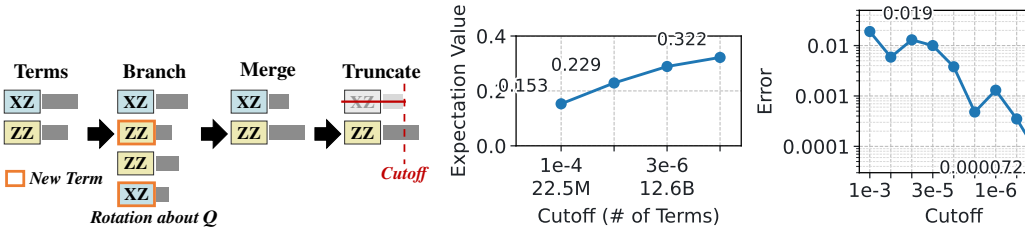
Branch, Merge, and Truncate. Each term of the list consists of a coefficient and a Pauli string, and the string maps to a bit vector with an x bit and a z bit for every qubit. With this bit vector, the product of two strings becomes the XOR of their bit vectors up to a phase. A rotation gate with generator Q keeps every term that commutes with Q and branches every anticommuting term P into P with $\cos\theta$ and $P \oplus Q$ with $\sin\theta$. These branches can double the number of terms at every rotation. The merge then sums the coefficients of the terms with equal strings, and the truncation removes the terms with a coefficient magnitude below the cutoff. Clifford gates map each string to one string and keep the number of terms, and noise channels scale the coefficients (Figure 1).

Term Growth with Accuracy. The cutoff of the truncation sets the accuracy and the number of terms together. On a 30-qubit circuit with an exact answer, the error decreases from 0.019 at cutoff 10^{-3} to 0.00007 at cutoff 10^{-7}. On the 127-qubit kicked Ising circuit, the expectation value moves from 0.153 at cutoff 10^{-4} to 0.229 at 10^{-5} and 0.322 at 10^{-6}. This movement requires lower cutoffs, and the circuit holds up to 22.5 million terms within a group of gates at cutoff 10^{-4}, 1.4 billion at 10^{-5}, and 100 billion at 10^{-6}. These peaks exceed the terms that remain after the group, since the rotations of a group branch the terms faster than its truncations remove them. With 40 bytes per term, the peak at cutoff 10^{-6} occupies 4 TB, the memory of about 1,000 GPUs.

Merge before Truncation. The truncation must see the merged coefficient of each string. Equal strings that arise from different parents carry parts of one coefficient, and each part can lie below the cutoff although their sum exceeds it. A truncation of these unmerged parts removes this sum and changes the answer. For example, the 127-qubit kicked Ising circuit at cutoff 10^{-4} yields 0.194-0.283 when the truncation sees unmerged parts, whereas the merged run yields 0.1531. The merge partner of a new term depends on the gate that creates it and not on the location of its parent. As a result,

Table 2. Distribution of state vector simulation and Pauli propagation

	State Vector	**Pauli Propagation**
Data	2^n amplitudes of fixed size	Terms that grow and shrink at every gate
Location of the data	Known from the index	Known only at run time
Work of a gate	Updates fixed positions	Creates, merges, and removes strings
Communication	Gates on the qubits of the index	Every gate that may create a remote string
Limit	Memory of 2^n amplitudes	Number of kept terms



(a) Operations of one gate (b) Kicked Ising value (c) 30-qubit TFIM error

Fig. 1. Operations and accuracy of Pauli propagation

a distributed run cannot place the merge partners on one GPU in advance, in contrast to a state vector that places each amplitude by its index (Table 2).

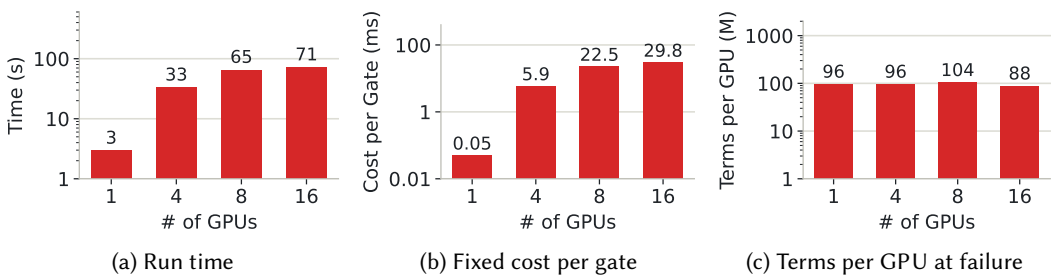
These two bottlenecks intensify at scale. The terms of an accurate run must spread over the GPUs of many nodes, whereas every equal string must meet its merge partners before each truncation. Existing systems run Pauli propagation on one device [4, 9], merge the terms across all GPUs after every gate [7], or fix the partition of the strings before the run [6]. A partition that follows both the gates and the live terms across nodes is not available in these systems.

2.2 Challenges in Distributed Pauli Propagation

Existing distributed systems merge the terms across all GPUs after every gate or partition the strings by rules fixed before the run [2, 6, 7]. We measure the first design with *cuPauliProp* v0.5.0 on the 127-qubit kicked Ising circuit and NVIDIA A100 GPUs, and the measurement exposes two challenges, a global step after every gate and a capacity bound by the GPU memory.

Global Step after Every Gate. *cuPauliProp* merges the terms of every gate in a global step, since a new term may merge with a term on any GPU. Each MPI rank applies the gate to its terms and calls allgather 22 times to merge and redistribute the terms with all other ranks, and 57% of the gates have no term to exchange. This global step adds a fixed cost of 5.9, 22.5, and 29.8 ms per gate on 4, 8, and 16 GPUs, against 0.05-0.8 ms on one GPU (Figure 2). As a result, a run with 22.5 million terms takes 2.9-3.2 s on one GPU and 33-48, 65, and 71 s on 4, 8, and 16 GPUs. The GPUs compute during 9% of this run, and the terms remain evenly spread at 1.001 times the mean on the largest rank. Thus, the cost of the global step, not an imbalance of the terms, slows the run as GPUs are added.

Capacity Bound by the GPU Memory. *cuPauliProp* also keeps all terms of every gate in GPU memory. Within a group of consecutive gates, the number of terms rises to 2.9-36 times its value at the start of the group in the seven circuits, and the merge requires working space next to these terms. As a result, *cuPauliProp* runs out of memory at 88-104 million terms per rank on 1 to 16



(a) Run time (b) Fixed cost per gate (c) Terms per GPU at failure

Fig. 2. Distributed mode of *cuPauliProp*

GPUs, and these kept terms occupy about 4 GB of the 36 GB that it uses on each GPU. A deferred merge reduces the global steps, but its unmerged copies exceed the memory of 16 GPUs on two of the seven circuits. A run without the global merge does not lift this bound and returns a wrong answer, 0.1675 on 4 GPUs and 0.189 on 16 GPUs against 0.1531, since its truncation discards parts of equal strings.

2.3 Opportunity in Parities Conserved by Windows

The two challenges require a partition that keeps every merge partner on the GPU of its parent without a global step. We examine where this partition exists with the generators of the seven circuits and with the terms of the kicked Ising circuit.

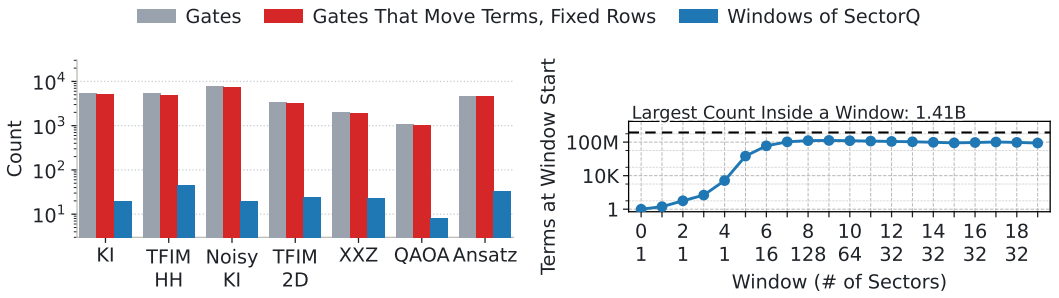
Parities Conserved by Windows of Gates. A rotation with generator Q changes a term P into $P \oplus Q$, and every parity r with $r \cdot Q = 0$ assigns the same value to both terms. A group of consecutive gates, a window, conserves every parity orthogonal to all of its generators, and these parities divide the terms into sectors that keep the merge partners together. We count the windows in the seven circuits under the rule that a window grows while at least n conserved parities remain (Table 3). The circuits with 1,100-7,960 gates form 8-45 windows, and a partition by window parities moves terms only at the window boundaries, 89-398 times fewer than the gates. In contrast, the whole circuit conserves at most two parities, and every term keeps the value of the observable on these parities. Thus, no parity divides the terms for the whole run, and the partition must change between windows.

Fixed Partitions Move Terms. Existing partitions fix their parities for the whole run and apply every gate to the terms [6]. Under these partitions, every gate that changes a fixed parity moves terms between processes. With 16 parts, rows over all bits move terms at 93-96% of the gates of the seven circuits, and rows over only the x bits or only the z bits move terms at 31-95% of the gates. On the kicked Ising circuit, random rows change the owner of 94% of the terms at a layer of two-qubit gates, and rows over lattice regions reduce this share to 68-89% with the largest of 16 parts at 1.65-3.37 times the mean. The library that implements these partitions keeps its rows for the whole run, since a changed row would move the owner of every term, and it lists the automatic choice of rows as an open problem [6].

Partitions that Follow the Run. The parities of each window follow its gates, and the balance of the sectors follows the live terms. On the kicked Ising circuit at cutoff 10^{-5}, the terms at the start of a window grow from 1 to 159 million over the run, and the first windows hold too few terms to divide (Figure 3). The terms within a window then rise to 2.9-36 times this start in the seven circuits, and the sectors of a balanced partition differ from window to window. As a result, only the live terms of the run determine the number of sectors and the parities that balance them. Thus,

Table 3. Windows and fixed partitions in seven circuits (16 parts, rows fixed without a Clifford frame)

Circuit	Gates	Windows	Gates per Window	Moving Gates (Random Rows)	Moving Gates (z Rows)
Kicked Ising	5,420	20	271	94%	50%
TFIM heavy-hex	5,420	45	120	93%	50%
Noisy kicked Ising	7,960	20	398	95%	50%
TFIM 11×11	3,410	24	142	94%	60%
XXZ 8×8	2,048	23	89	94%	62%
QAOA	1,100	8	138	94%	60%
Ansatz	4,784	34	141	96%	95%



(a) Gates and windows

(b) Terms at window starts

Fig. 3. Windows of gates in seven circuits

Table 4. Requirements of distributed Pauli propagation

#	Requirement	Evidence
R1	Merge every equal string before each truncation	Unmerged parts change the answer
R2	No coordination among GPUs during the gates	The fixed cost per gate grows with the GPUs
R3	No accumulation of unmerged copies	A deferred merge exceeds the GPU memory
R4	A partition recomputed during the run	Fixed rows move terms at 31-96% of the gates
R5	A capacity beyond the GPU memory	Kept terms fill 4 of 36 GB on one GPU
R6	No loss on small problems	0.42 s on one GPU becomes 32-47 s on four

distributed Pauli propagation requires a partition that the system recomputes for every window from its gates and from the live terms during the run.

These observations define six requirements for distributed Pauli propagation, listed in Table 4. A partition that keeps every merge partner in the part of its parent satisfies the first three requirements together.

3 SectorQ Design

3.1 Overall Architecture

Window Loop.

Data Placement.

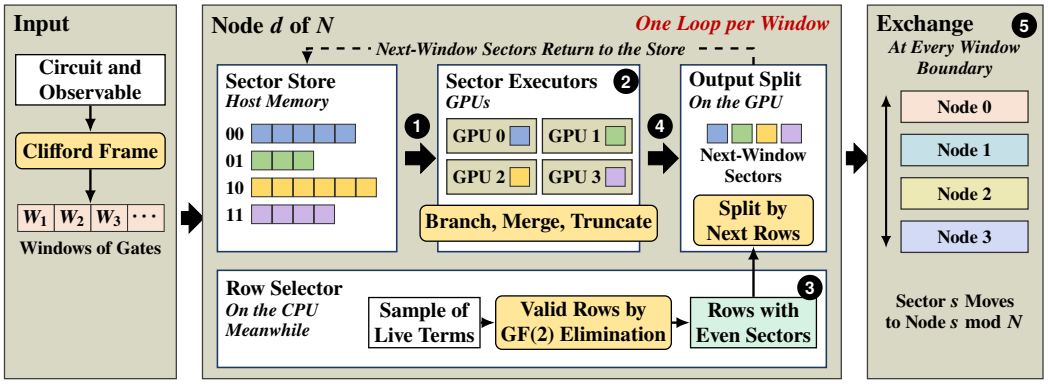
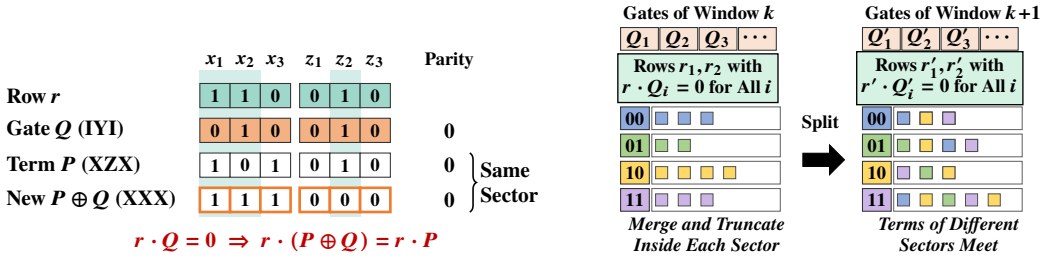


Fig. 4. Overall architecture of *SectorQ*



(a) Parity conserved by a gate

(b) Sectors of two windows

Fig. 5. Window-conserved parity sectors

3.2 Window-Conserved Parity Sectors

Parity of a Term.
Rows Conserved by a Window.
Merge Locality.

3.3 Run-Time Row and Sector Selection

Sampling the Live Terms.
Overlap with the Running Window.
Number of Sectors.

3.4 Host-Memory Sector Store

Chunked Storage.
Sector Runs on GPUs.
Splitting Oversized Sectors.

3.5 Exchange at Window Boundaries

Sector Ownership.
Exchange in Rounds.
Checkpoints.

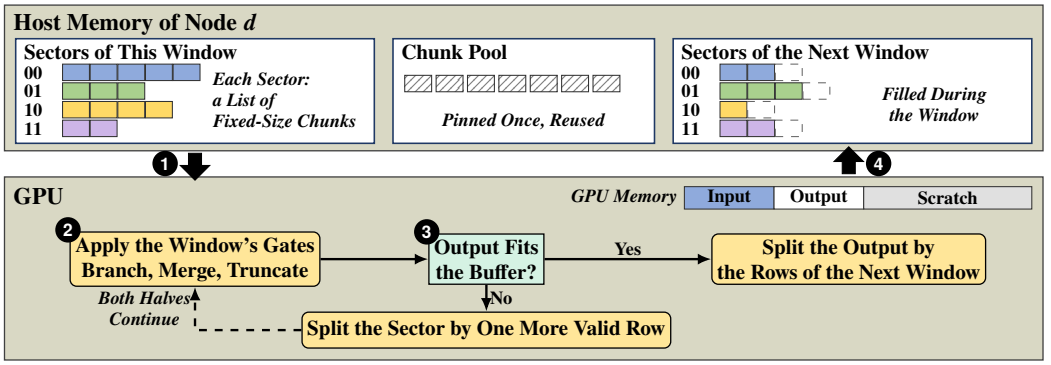


Fig. 6. Host memory and one GPU during a window

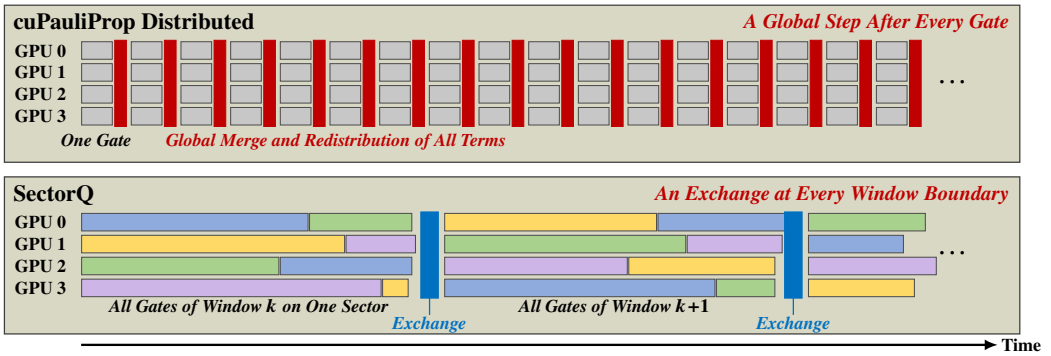


Fig. 7. Global steps of *cuPauliProp* and *SectorQ* on four GPUs

3.6 Clifford Frame

3.7 Implementation

4 Evaluation

4.1 Evaluation Setup

Hardware Specification.
Evaluation Scope.
Circuits.
Baselines.

4.2 Single-Node Performance

Comparison with GPU Implementations.
Comparison with CPU Implementations.

4.3 Performance across Circuits on 16 GPUs

Simulation Time.
Time per Term.
Agreement of Answers.

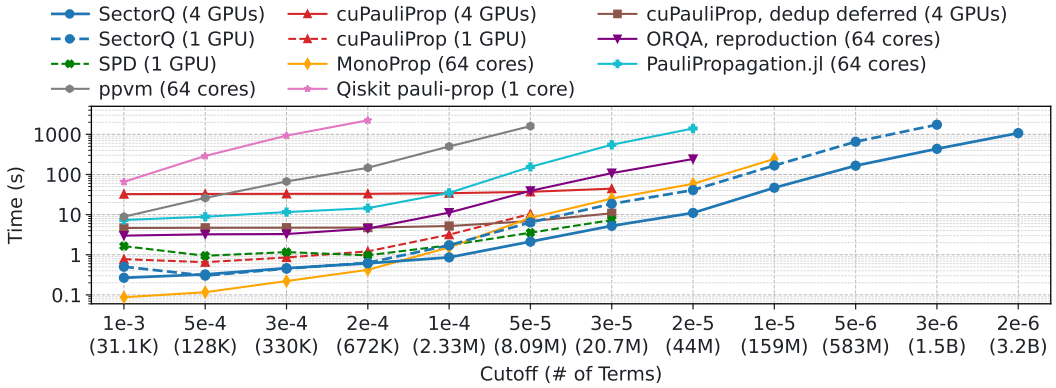


Fig. 8. Simulation time on one node (4 GPUs)

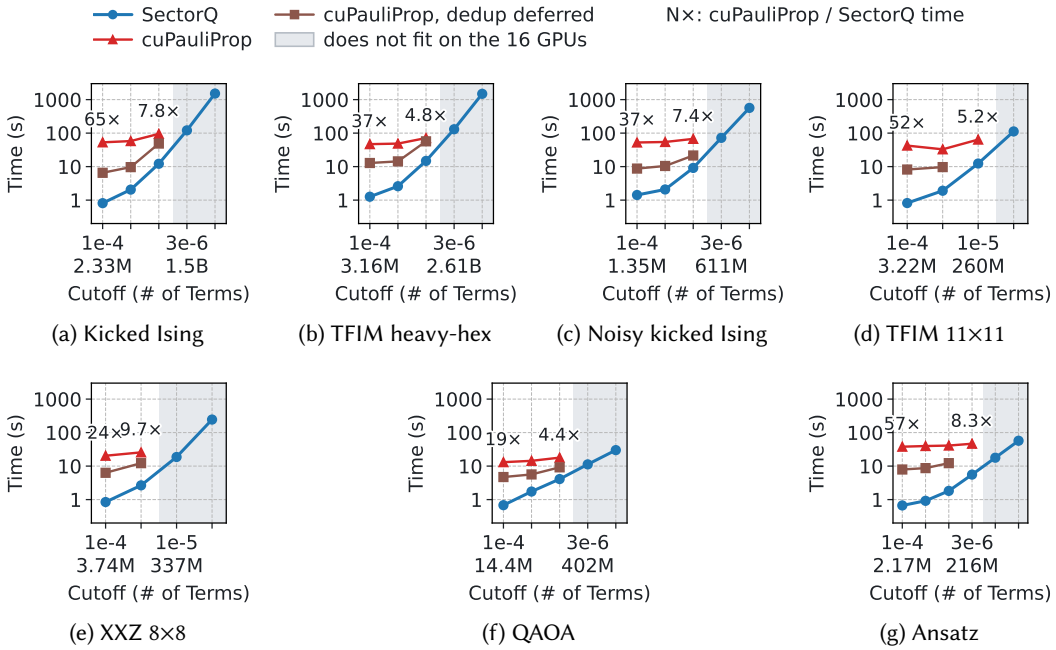


Fig. 9. Simulation time of seven circuits on 16 GPUs

4.4 Comparison with Fixed Linear Partitions

Same Truncation Rule.
Fixed versus Recomputed Rows.

4.5 Multi-Node Scalability

Weak Scalability.
Strong Scalability.
Three Circuits on 128 GPUs.

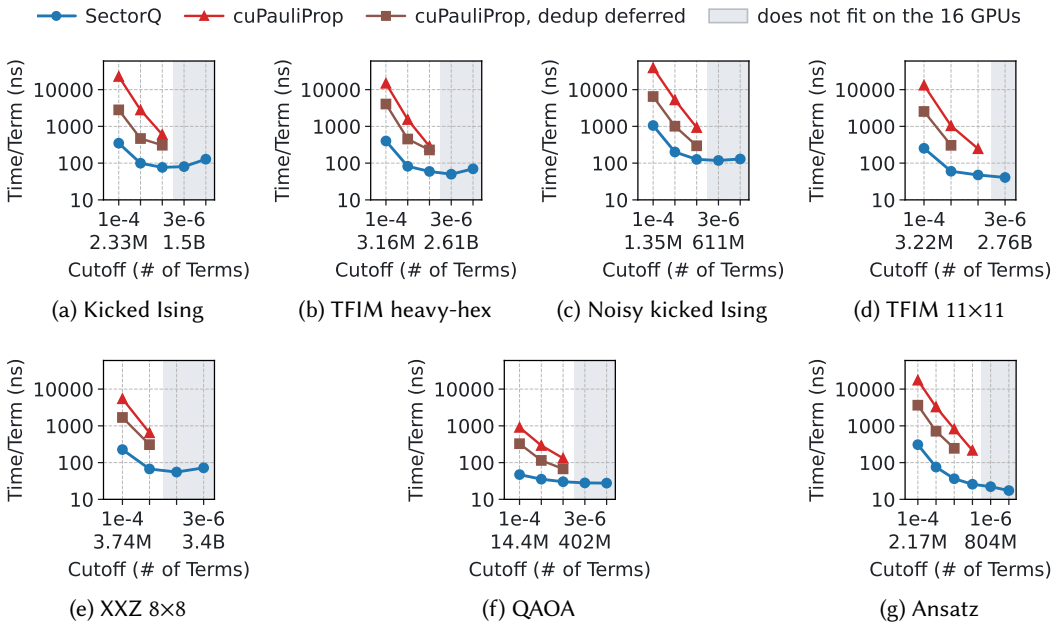


Fig. 10. Time per term of seven circuits on 16 GPUs

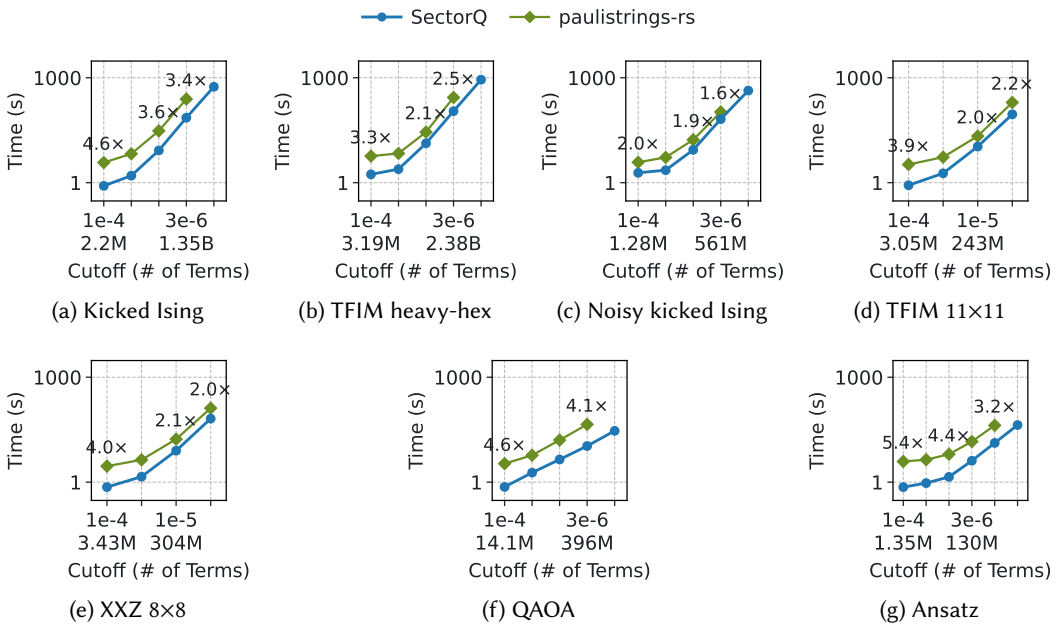


Fig. 11. *SectorQ* and fixed linear partitions on 16 GPUs

4.6 Circuit Size and Depth

Number of Qubits.

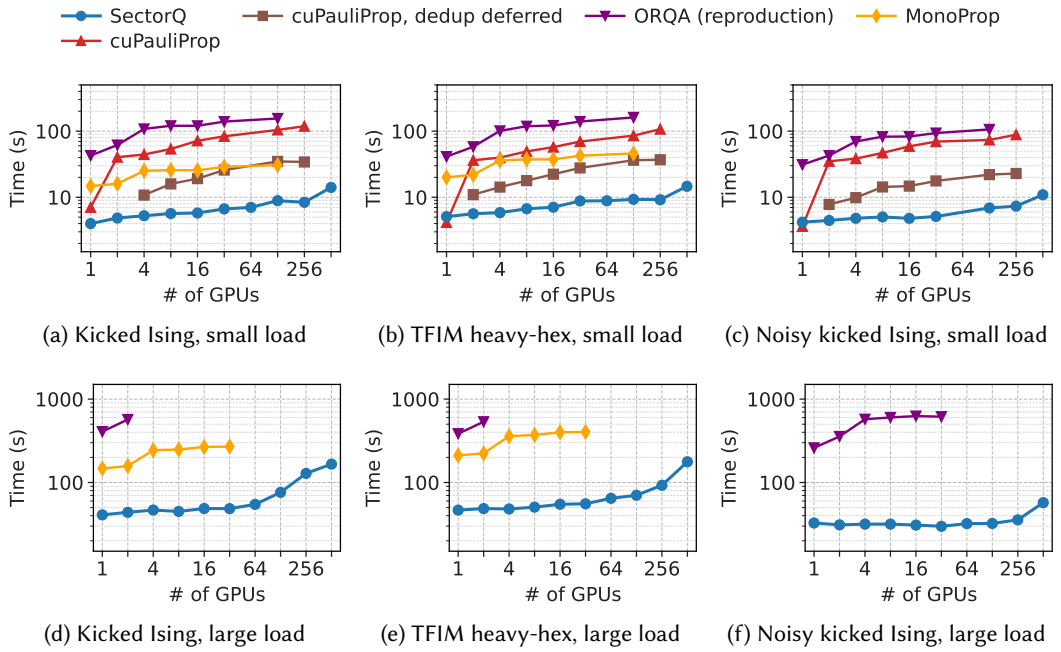


Fig. 12. Weak scaling from 1 to 512 GPUs

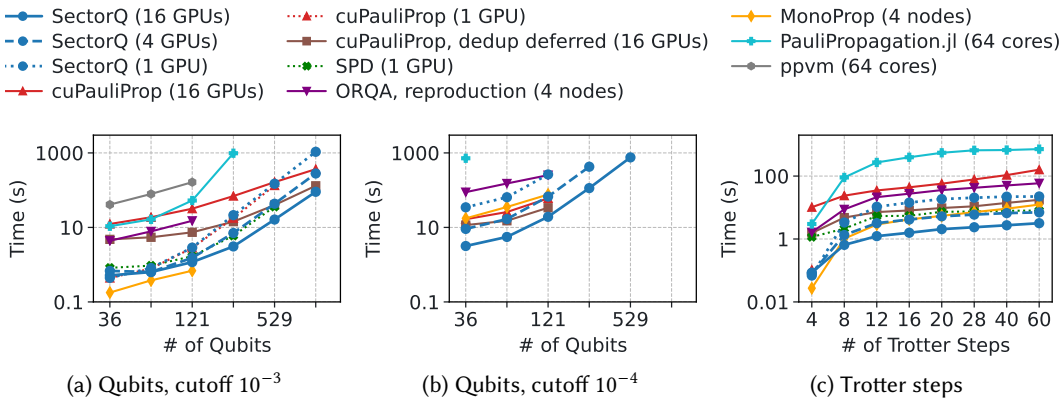


Fig. 13. Simulation time by circuit size and depth

Circuit Depth.

4.7 Extreme Scale-out

Largest Finished Problems.
Host Memory Use.

4.8 Time Analysis

Where the Time Goes.
Contribution of Each Element.

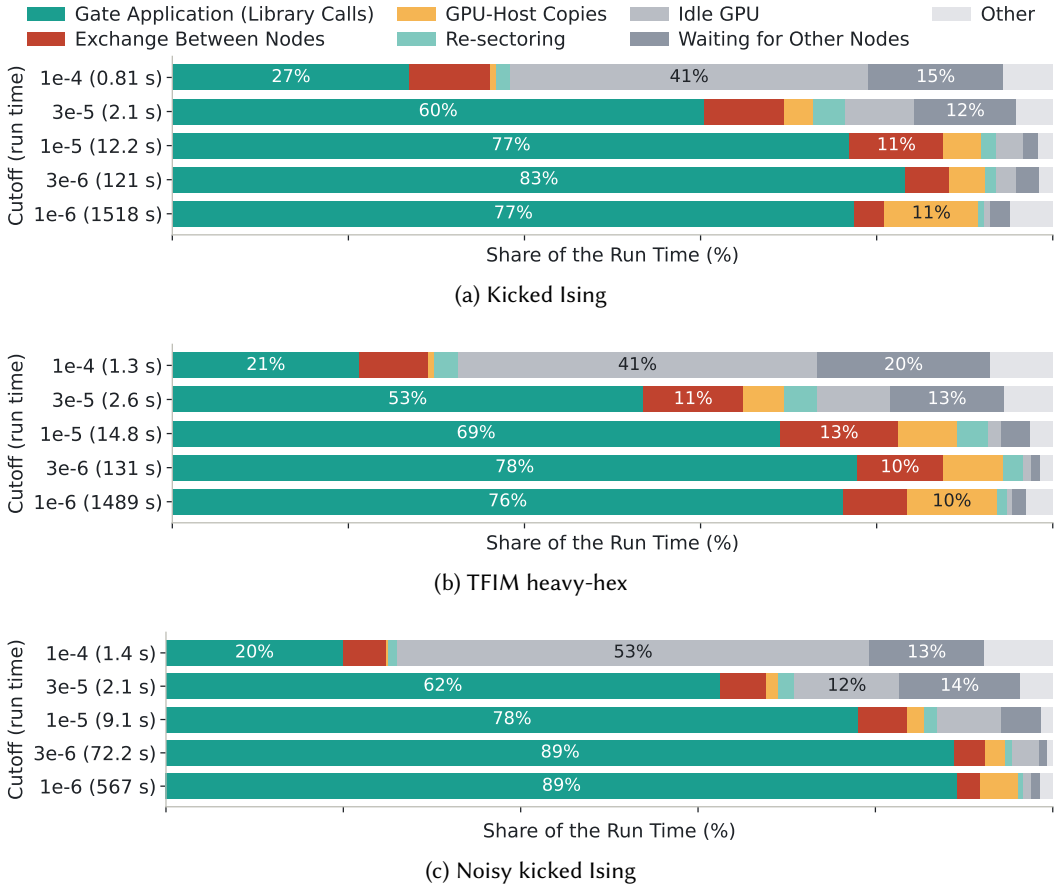


Fig. 14. Run time of *SectorQ* by activity on 16 GPUs

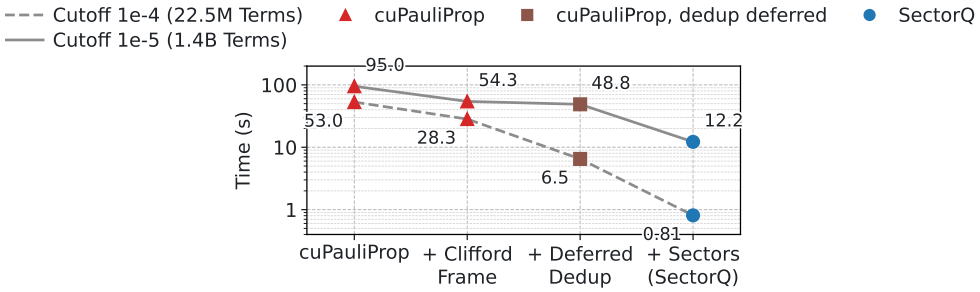
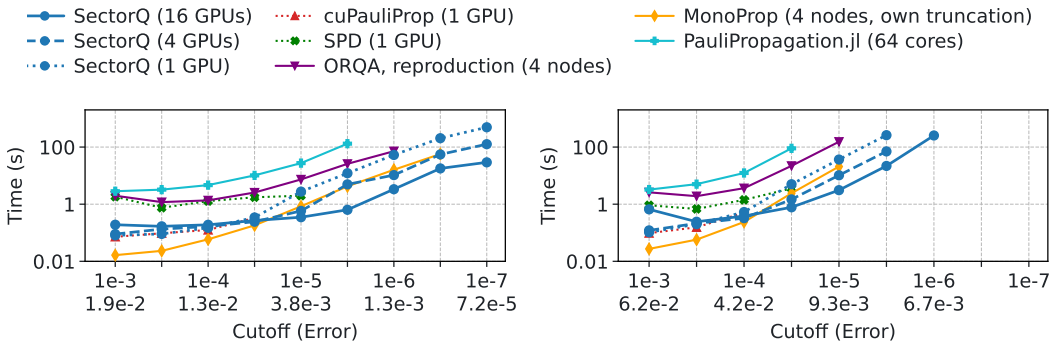


Fig. 15. Run time as each element is added on 16 GPUs

4.9 Accuracy

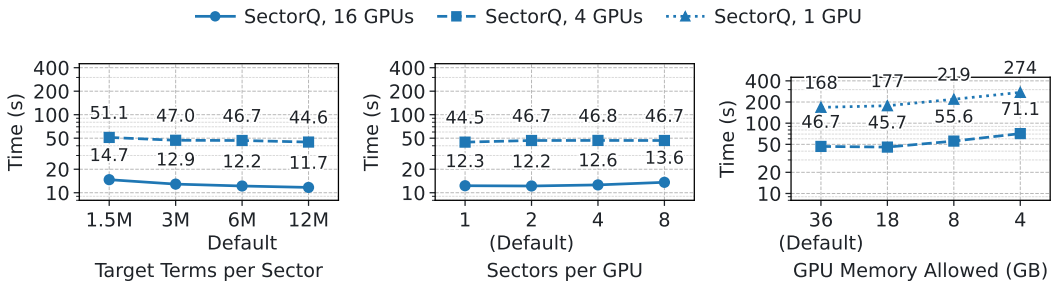
Exact Reference.

Convergence of Kicked Ising.



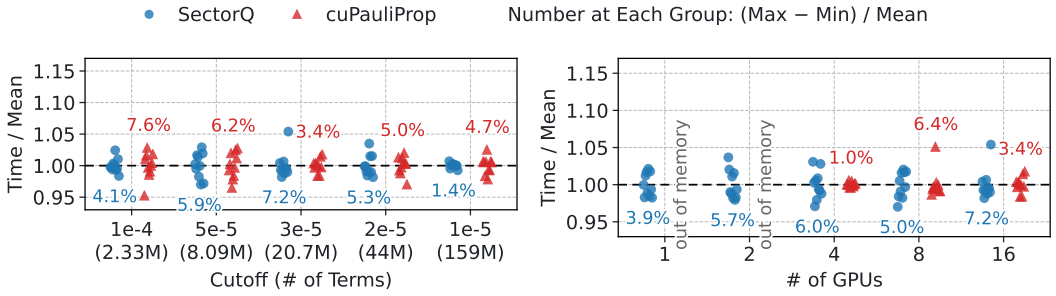
(a) 4 Trotter steps (b) 6 Trotter steps

Fig. 16. Simulation time and error of a 30-qubit TFIM



(a) Target terms per sector (b) Sectors per GPU (c) GPU memory allowed

Fig. 17. Run time by parameter on kicked Ising at cutoff 10^{-5}



(a) 16 GPUs by problem size (b) Cutoff $3{\times}10^{-5}$ by GPU count

Fig. 18. Run time of ten repeated runs

4.10 Sensitivity to Parameters

4.11 Performance Variability

5 Related Work

Pauli Propagation Simulators.

Partitioning by Conserved Quantities.

Distributed Quantum Circuit Simulation.

6 Conclusion

In this paper, we design and implement *SectorQ*, a Pauli propagation system for GPU clusters that partitions the terms by the parities that each window of gates conserves. By recomputing these parities for every window from the generators of its gates and the live terms, we confine the merging of terms to one sector during a window and hold the sectors in host memory. We evaluate *SectorQ* with seven circuits on up to 512 NVIDIA A100 GPUs, reducing the simulation time of *cuPauliProp* by 4.4-9.7× on 16 GPUs and increasing the largest finished problem by 140× on 512 GPUs through exchanges at window boundaries and sectors in host memory.

References

[1] [n. d.]. MonoProp. TODO: source and authors.
[2] 2025. ORQA. arXiv:2506.13241. TODO: verify authors and title.
[3] Frank Arute et al. 2019. Quantum supremacy using a programmable superconducting processor. *Nature* 574 (2019), 505–510.
[4] Tomaš Begušić, Johnnie Gray, and Garnet Kin-Lic Chan. 2024. Fast and converged classical simulations of evidence for the utility of quantum computing before fault tolerance. *Science Advances* (2024). TODO: verify volume and pages.
[5] Youngseok Kim, Andrew Eddins, Sajant Anand, Ken Xuan Wei, Ewout van den Berg, Sami Rosenblatt, Hasan Nayfeh, Yantao Wu, Michael Zaletel, Kristan Temme, and Abhinav Kandala. 2023. Evidence for the utility of quantum computing before fault tolerance. *Nature* 618 (2023), 500–505.
[6] lkdvos. 2026. paulistrings-rs. https://github.com/lkdvos/paulistrings-rs. GitHub repository of the user lkdvos, commit 99c3436. TODO: verify authors.
[7] NVIDIA. 2025. cuPauliProp: Pauli propagation in the cuQuantum SDK. https://docs.nvidia.com/cuda/cuquantum/. Version 0.5.0. TODO: verify title and URL.
[8] John Preskill. 2018. Quantum Computing in the NISQ era and beyond. *Quantum* 2 (2018), 79.
[9] Manuel S. Rudolph et al. 2025. PauliPropagation: A Computational Framework for Simulating Quantum Systems. arXiv:2505.21606. TODO: verify.
[10] Joseph Tindall, Matthew Fishman, E. Miles Stoudenmire, and Dries Sels. 2024. Efficient Tensor Network Simulation of IBM's Eagle Kicked Ising Experiment. *PRX Quantum* (2024). TODO: verify volume and article number.